

RELATÓRIO – TAREFA 1_A

Grafo da base Iris com matriz de adjacências em C

1. IDENTIFICAÇÃO DA EQUIPE

Integrante	Curso e instituição
Nicolas Oliveira Veiga	Bacharelado em Ciência da Computação – UDESC (CCT), Joinville/SC
Guilherme Wippel Kormann	Bacharelado em Ciência da Computação – UDESC (CCT), Joinville/SC

2. IDENTIFICAÇÃO DA TAREFA

Tarefa 1_A – TEG: grafo da base Iris. O trabalho pede um programa em C, executável no Ubuntu, que implemente um TAD *Grafo* por matriz de adjacências e o construa a partir da base Iris (150 observações de três espécies, 50 de cada, com quatro medidas em centímetros). Cada observação é um vértice (c1 a c150) e dois vértices são ligados quando suas observações são suficientemente parecidas. O enunciado completo está nos PDFs da tarefa 1 no Moodle; os requisitos que o programa atende são:

- **carga primária:** ler o CSV da base, ignorar a coluna da espécie e usar as quatro medidas, calcular a tabela de distâncias euclidianas (DE), normalizá-la (DEN) e gerar a matriz de adjacências com limiar 0,3 sobre a DEN;
- **persistência:** gravar o grafo em CSV, com um cabeçalho de metadados e a matriz, e recarregá-lo depois sem reler a base e sem recalculando DE e DEN;
- **cabeçalho do CSV:** total de vértices; maior e menor DE e DEN, cada um com o par de vértices que o determinou; graus máximo e mínimo; tipo do grafo (simples ou multigrafo, número de laços, de arestas múltiplas e de arestas); número de componentes conexos, com o tamanho e o vértice inicial de cada um;
- **visualização:** exibir o grafo em três dimensões, de forma interativa.

3. EXPLICAÇÃO CONCEITUAL DA SOLUÇÃO

A solução é um programa de terminal com menu, dividido em um módulo de grafo (TAD), o programa principal e um script Python para a visualização. A Tabela 1 resume os arquivos.

Tabela 1 – Arquivos do projeto

Arquivo	Função
<code>grafo.h / grafo.c</code>	TAD <i>Grafo</i> (tipo opaco, matriz de adjacências), carga primária, persistência em CSV, resumo e verificação
<code>main.c</code>	Menu interativo no terminal; usa o TAD apenas pelas funções de <code>grafo.h</code>
<code>Makefile</code>	<code>make</code> compila ; <code>make teste</code> roda carga, salvamento, recarga e verificação pelo menu
<code>visualizar_grafo.py</code>	Visualização 3D interativa (Plotly), chamada pela opção 8 do menu
<code>original_IrisDataset.csv</code>	Base Iris de entrada (150 observações)
<code>grafo_iris.csv</code>	Grafo persistido (exemplo gerado com limiar 0,3)
<code>grafo_iris_3d.html</code>	Visualização 3D exportada pelo script

Fonte: elaborado pelos autores.

3.1 Entrada

A entrada principal é `original_IrisDataset.csv`: uma linha de cabeçalho (`variety, sepal.length, sepal.width, petal.length, petal.width`) e 150 linhas com a espécie (Setosa, Versicolor e Virginica, 50 de cada) e as quatro medidas. A primeira coluna é ignorada; as quatro seguintes formam o ponto de cada vértice em um espaço de dimensão 4. A ordem das linhas define os vértices: c1 a c50 são Setosa, c51 a c100 Versicolor e c101 a c150 Virginica.

As demais entradas vêm do menu: o nome dos arquivos (ENTER aceita o valor padrão entre colchetes) e os vértices consultados, escritos como `c12` ou `12`. Para recarregar um grafo, a entrada é o próprio CSV de persistência. O limiar da DEN é fixo em 0,3, como pede o enunciado (constante `LIMIAR_PADRAO` em `grafo.h`).

3.2 Estrutura de dados

O tipo *Grafo* é opaco: `grafo.h` apenas o declara e `grafo.c` o define, de modo que `main.c` só o manipula por funções como `grafo_criar`, `grafo_adicionar_aresta`, `grafo_grau`, `grafo_multiplicidade`,

`grafo_carregar_iris_csv`, `grafo_salvar_csv`, `grafo_carregar_csv` e `grafo_verificar`. Internamente o grafo guarda o número de vértices n , um vetor de inteiros $n \times n$ com a matriz de adjacências (linha a linha) e uma estrutura `GrafoInfo` com os metadados do cabeçalho.

Cada posição da matriz guarda o *número de arestas* entre dois vértices, e não apenas 0 ou 1. Assim o mesmo TAD representa grafos simples e multigrafos, e um laço (posição da diagonal) conta 2 no grau do vértice. O grafo é não direcionado, portanto a matriz é simétrica. Para $n = 150$ a matriz ocupa 22.500 inteiros (cerca de 88 KiB). Internamente os vértices vão de 0 a $n-1$; na tela e no CSV aparecem como $c1$ a cn .

3.3 Processo

Carga primária (`grafo_carregar_iris_csv`). Segue cinco passos:

1. **Leitura e validação.** Cada linha precisa ter exatamente cinco colunas. A primeira linha não numérica é tratada como cabeçalho; linhas malformadas, valores não numéricos e bases com menos de duas observações geram mensagem de erro.
2. **Tabela DE.** Distância euclideana entre todos os pares de observações, calculada uma vez por par $i < j$ e espelhada na tabela. O programa guarda a maior e a menor DE com o par de vértices; em caso de empate vale o primeiro par encontrado na ordem $i < j$.

$$DE(v_i, v_j) = \sqrt{(\sum_{k=1 \dots 4} (x_{ik} - x_{jk})^2)}$$

3. **Tabela DEN.** Normalização min-max da tabela, com mínimo e máximo calculados sobre os pares distintos. O resultado fica em $[0, 1]$: a maior DEN é sempre 1 e a menor é sempre 0.

$$DEN(v_i, v_j) = [DE(v_i, v_j) - DE_{min}] / [DE_{max} - DE_{min}]$$

4. **Matriz de adjacências.** Há aresta entre dois vértices distintos quando a DEN é menor ou igual ao limiar. Como cada par é visitado uma vez ($i < j$), não surgem laços nem arestas múltiplas: o grafo é simples.

$$\{v_i, v_j\} \in A \Leftrightarrow i < j \text{ e } DEN(v_i, v_j) \leq 0,3$$

5. **Propriedades estruturais** (`calcular_estrutura`). Graus (máximo e mínimo, com um vértice que os atinge), laços, arestas múltiplas e número de arestas saem da matriz. Os componentes conexos são encontrados por busca em profundidade iterativa, com pilha explícita, em $O(n^2)$ por causa da matriz;

ficam ordenados do maior para o menor e cada um é identificado pelo seu menor vértice.

Persistência. `grafo_salvar_csv` grava o cabeçalho (uma linha `chave, valor` por propriedade), a linha `matriz_adjacencias` e as n linhas da matriz. `grafo_carregar_csv` reconstrói o grafo só a partir desse arquivo: confere a assinatura do formato, exige todas as chaves, valida as dimensões e a simetria da matriz e rejeita arquivos truncados ou inválidos. A opção 5 do menu (`grafo_verificar`) recalcula as propriedades a partir da matriz e compara com o cabeçalho, apontando qualquer divergência.

Interface. `main.c` oferece nove opções: carga primária, salvar, recarregar, resumo, verificação cabeçalho $\times$ matriz, consulta de vértice (grau e vizinhos), teste de adjacência entre dois vértices, visualização 3D e sair. O programa pede confirmação antes de sobrescrever um arquivo e antes de sair com o grafo não salvo, e trata fim de entrada (EOF) sem travar.

3.4 Saída

O programa produz três saídas. A primeira é o resumo no terminal, com todos os itens do cabeçalho. A segunda é o arquivo CSV de persistência, cujo cabeçalho no grafo gerado é:

```
grafo_iris_csv,versao,1
total_vertices,150
limiar_DEN,0.3000000000
maior_DE,7.0851958336,c14,c119
menor_DE,0.0000000000,c102,c143
maior_DEN,1.0000000000,c14,c119
menor_DEN,0.0000000000,c102,c143
grau_maximo,93,c78
grau_minimo,30,c119
tipo_grafo,simples
numero_lacos,0
numero_arestas_multiplas,0
numero_arestas,5250
numero_componentes,1
componente,1,tamanho,150,vertice_inicial,c1
matriz_adjacencias
0,1,1,1,1,1,... (150 linhas com 150 valores cada)
```

A terceira saída é a visualização 3D (opção 8), gerada por `visualizar_grafo.py` a partir do CSV do grafo e da base. Como cada observação tem quatro medidas e o gráfico tem três eixos, o padrão é projetar os pontos nos três primeiros componentes principais (PCA, calculada por decomposição em valores singulares, sobre as medidas em cm e sem padronização, coerente com a distância euclidiana do grafo).

Esses três componentes explicam 99,48% da variância (92,46%, 5,31% e 1,71%). A alternativa `--eixos atributos --attrs 0 2 3` usa três das medidas originais. Os vértices são coloridos por espécie e, ao passar o mouse, aparecem o grau e as medidas; o gráfico gira com o mouse.

3.5 Decisões de projeto

- **Empates** em maior e menor DE ou DEN: vale o primeiro par encontrado na ordem $i < j$.
- **Menor DE igual a 0**: c102 e c143 são observações idênticas na base. Por isso a menor DE e a menor DEN são 0, e os dois vértices ficam adjacentes.
- **Limiar fixo em 0,3** no menu, como no enunciado. A função de carga recebe o limiar como parâmetro, então outro valor exige apenas mudar a constante e recompilar.
- **Robustez de entrada**: arquivos ausentes, formatos inválidos e vértices fora do intervalo 1 a 150 produzem mensagem de erro e o menu continua funcionando.

4. INSTRUÇÕES SOBRE A COMPILAÇÃO

O projeto *não usa IDE nem arquivo de projeto*: compila-se pela linha de comando com o `gcc`, no padrão C99. A única biblioteca ligada é a de matemática (`libm`, opção `-lm`), necessária no Linux por causa de `sqrt`. Não há bibliotecas externas em C. Na pasta que contém `main.c`, `grafo.c` e `grafo.h`:

```
gcc -std=c99 -Wall -Wextra -pedantic -O2 -o grafo_iris main.c grafo.c -lm
# ou, com make instalado:
make
```

A compilação foi testada em Ubuntu 24.04 com gcc 13.3.0 e terminou sem nenhum aviso, mesmo com `-Wall -Wextra -pedantic`. Para executar, na pasta do projeto (que precisa conter `original_IrisDataset.csv` e `visualizar_grafo.py`):

```
./grafo_iris
# teste automatizado (carga, salvar, recarregar e verificar pelo menu):
make teste
```

O fluxo típico é opção **1** (carga), **2** (salvar) e **8** (visualizar); em outra sessão, a opção **3** recarrega o grafo sem reler a base. A opção **8** chama `python3`

`visualizar_grafo.py`, que exige o Python 3 com duas bibliotecas, instaladas uma única vez:

```
pip install plotly numpy
```

As demais opções do menu não dependem de Python. O arquivo `grafo_iris_3d.html` referencia o Plotly.js por CDN, portanto abri-lo exige conexão com a internet.

5. DESCRIÇÃO E ANÁLISE DOS RESULTADOS

5.1 Resultado da carga primária

A Tabela 2 reúne os itens do cabeçalho para o grafo gerado com limiar 0,3. O grafo é simples, tem 150 vértices e 5.250 arestas e é conexo. Das 11.175 combinações de dois vértices, 5.250 (47,0%) viram aresta.

Tabela 2 – Propriedades do grafo da base Iris (limiar DEN $\leq 0,3$)

Item	Propriedade	Resultado
i	Total de vértices	150
ii	Maior DE	7,0852 (c14, c119)
iii	Menor DE	0,0000 (c102, c143)
iv	Maior DEN	1,0000 (c14, c119)
v	Menor DEN	0,0000 (c102, c143)
vi	Grau máximo / grau mínimo	93 (c78) / 30 (c119)
vii	Tipo do grafo	Simple (0 laços, 0 arestas múltiplas)
vii	Número de arestas	5.250
viii	Componentes conexos	1 (150 vértices, a partir de c1)

Fonte: saída do programa (opção 4 do menu).

O par mais distante, c14 (Setosa) e c119 (Virginica), tem DE de 7,0852 cm. O vértice c119 também tem o menor grau (30): é a observação mais isolada da base. O par mais próximo, c102 e c143, é de observações idênticas. Como a menor DE é 0, a regra $DEN \leq 0,3$ equivale a $DE \leq 0,3 \times 7,0852 \approx 2,1256$ cm.

5.2 Validação

Os resultados foram conferidos de quatro formas:

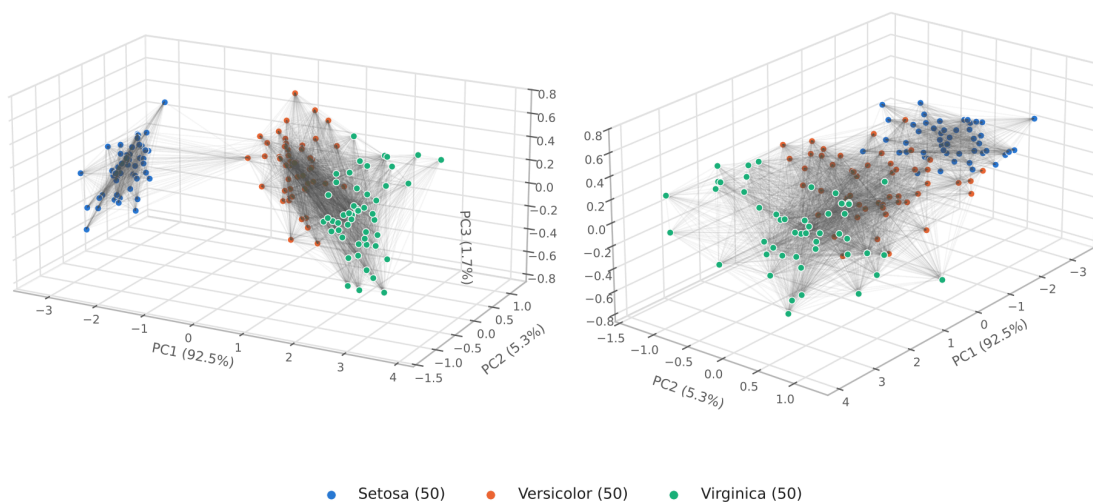
- **Recálculo independente.** Um script Python com numpy refez DE, DEN, a matriz, os extremos, os graus e os componentes. A matriz é idêntica à do programa em C e todos os valores do cabeçalho coincidem.
- **Salvar e recarregar.** O grafo recarregado do CSV, sem rerear a base, produz o mesmo resumo, e a opção 5 responde `OK (coerente)`.
- **Ferramentas de análise.** Uma execução do menu (carga, salvar, recarregar, verificar e consultar vértice) compilada com AddressSanitizer e UBSan terminou sem erros de memória nem comportamento indefinido.
- **Entradas inválidas.** Vértice fora de 1 a 150 e arquivo inexistente geram mensagem de erro, e o menu continua funcionando.

5.3 Estrutura do grafo por espécie

A Figura 1 mostra o grafo projetado nos três componentes principais, com os vértices coloridos por espécie, em duas vistas. A Tabela 3 conta as arestas dentro de cada espécie e entre espécies.

Figura 1 – Grafo Iris (150 vértices, 5.250 arestas) projetado em 3 componentes principais

Grafo Iris: 150 vértices, 5250 arestas ($DEN \leq 0,3$), projeção nos 3 componentes principais



Fonte: elaborado pelos autores, com a mesma projeção PCA do script de visualização (renderizada com matplotlib para este relatório).

Tabela 3 – Arestas por par de espécies e grau dos vértices (limiar 0,3)

Espécie	Setosa	Versicolor	Virginica	Grau médio (mín.–máx.)
Setosa	1.222	54	0	50,0 (47–53)
Versicolor	54	1.176	1.681	81,7 (46–93)

Espécie	Setosa	Versicolor	Virginica	Grau médio (mín.–máx.)
Virginica	0	1.681	1.117	78,3 (30–92)

Fonte: elaborado pelos autores. A diagonal conta as arestas dentro da espécie.

Setosa forma um bloco quase isolado. Das 1.225 combinações entre vértices da espécie, 1.222 viram aresta: todo vértice de Setosa é adjacente a praticamente todos os outros 49, e o grau fica entre 47 e 53. Por exemplo, c1 tem grau 50, com os vizinhos c2 a c50 e c99. Setosa liga-se ao resto do grafo por apenas 54 arestas, todas com Versicolor; não há nenhuma com Virginica.

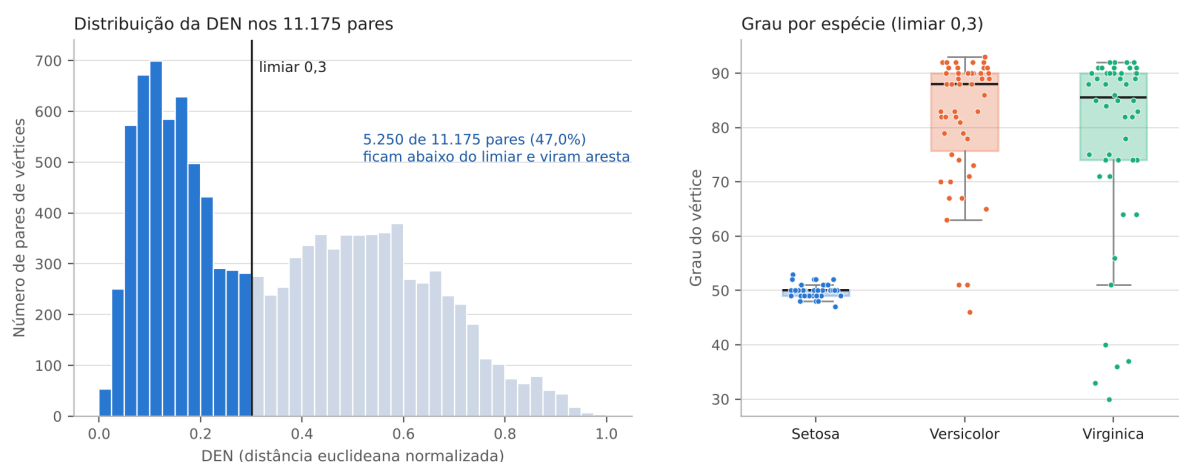
Versicolor e Virginica se sobrepõem. Há 1.681 arestas entre as duas espécies (67% das 2.500 combinações possíveis), mais do que dentro de Versicolor (1.176) ou de Virginica (1.117). Os graus são altos e variam muito, de 30 a 93. Essa sobreposição é coerente com o fato de as duas espécies terem medidas parecidas. Já os vértices de menor grau, como c119, são observações de Virginica afastadas do centro do grupo.

Como as 54 arestas Setosa–Versicolor são as únicas que ligam Setosa ao resto, o grafo só é conexo por causa delas. A seção seguinte mostra o que acontece com limiares menores.

5.4 Efeito do limiar

A Figura 2 mostra a distribuição da DEN nos 11.175 pares e o grau por espécie. A Tabela 4 repete o cálculo para outros limiares em um script auxiliar em Python, já que o programa em C usa o limiar fixo de 0,3.

Figura 2 – Distribuição da DEN e grau dos vértices por espécie



Fonte: elaborado pelos autores, a partir da matriz gerada pelo programa.

Tabela 4 – Sensibilidade do grafo ao limiar (cálculo auxiliar)

Limiar	Arestas	Densidade	Grau mín.	Grau máx.	Componentes (tamanhos)
0,05	304	2,7%	0	17	38 (maior: 45)
0,10	1.549	13,9%	0	41	4 (97, 50, 2, 1)
0,15	2.833	25,4%	5	60	2 (100, 50)
0,20	3.959	35,4%	10	78	2 (100, 50)
0,25	4.682	41,9%	19	88	1 (150)
0,30	5.250	47,0%	30	93	1 (150)
0,40	6.330	56,6%	51	135	1 (150)

Fonte: elaborado pelos autores. A linha destacada é o limiar da tarefa.

Com limiar 0,20 (ou 0,15) o grafo tem dois componentes: um com os 50 vértices de Setosa e outro com os 100 de Versicolor e Virginica. Entre 0,20 e 0,25 surgem as primeiras arestas Setosa–Versicolor e o grafo passa a ser conexo, o que se mantém em 0,30. Com limiares muito baixos (0,05 e 0,10) aparecem vértices isolados e vários componentes pequenos. O limiar escolhido, 0,3, equilibra os dois extremos: o grafo é conexo, mas ainda deixa fora mais da metade dos pares (53,0%).

5.5 Considerações finais

O programa atende ao que a tarefa pede: constrói o grafo da base Iris por matriz de adjacências, calcula todas as propriedades do cabeçalho, persiste e recarrega o grafo sem recalculando DE e DEN e o exibe em 3D. Os resultados foram confirmados por uma implementação independente. Quanto às limitações: a matriz de adjacências ocupa $O(n^2)$ de memória, o que é adequado para 150 vértices, mas não para grafos muito grandes; a DEN depende do par de observações mais distante (c14, c119), de modo que a inclusão de um ponto extremo muda todas as DEN; e a visualização exige Python com Plotly e, no arquivo HTML exportado, acesso à internet.

REFERÊNCIAS

PLOTLY. *3D scatter plots in Python*. Disponível em: <<https://plotly.com/python/3d-scatter-plots/>>.

PLOTLY. *3D line plots in Python*. Disponível em: <<https://plotly.com/python/3d-line-plots/>>.